Tema 5. Polimorfismo

1. Brevemente, ¿qué es el "polimorfismo" y para qué sirve en programación orientada a objetos? ¿qué es la "sobreescritura" de métodos?

Respuesta

El polimorfismo es un principio fundamental de la programación orientada a objetos que permite que un mismo método o función se comporte de manera diferente dependiendo del objeto que lo invoque. En términos simples, significa "muchas formas" y se utiliza para escribir código más flexible y reutilizable. Por ejemplo, se puede tener una referencia de tipo genérico (como una clase base) que apunte a objetos de diferentes subclases, y al invocar un método, se ejecutará la implementación específica de la subclase correspondiente.

La sobreescritura de métodos es una característica que permite a una subclase proporcionar una implementación específica de un método que ya está definido en su clase base. Esto se logra utilizando el mismo nombre, parámetros y tipo de retorno, pero redefiniendo el comportamiento del método en la subclase. Es una herramienta clave para implementar polimorfismo, ya que permite que las subclases personalicen el comportamiento heredado.

2. ¿En qué consiste la "ligadura dinámica" o "enlace tardío"? ¿qué relación tiene con el polimorfismo? ¿hay que indicarlos explícitamente al programar o depende esto del lenguaje? Compara C++ y Java. Indicalo después también para Python.

Respuesta

La ligadura dinámica, también conocida como enlace tardío, es el proceso mediante el cual se determina en tiempo de ejecución qué implementación de un método debe ejecutarse. Esto ocurre

cuando se utiliza polimorfismo, ya que el método invocado depende del tipo real del objeto al que apunta la referencia, y no del tipo de la referencia en sí.

En Java, la ligadura dinámica es automática para los métodos no estáticos. No es necesario indicarlo explícitamente, ya que el lenguaje está diseñado para soportar polimorfismo de manera predeterminada. En C++, por otro lado, se debe usar la palabra clave `virtual` en la declaración del método en la clase base para habilitar la ligadura dinámica. Si no se utiliza `virtual`, el enlace será estático (en tiempo de compilación). En Python, la ligadura dinámica también es automática debido a su naturaleza dinámica y no requiere configuraciones adicionales.

3. Pon un ejemplo sencillo en Java, de un `Soldado`, con un método `saluda`, con dos subclases: `Zapador` y `Artillero`, donde `Zapador` sobreescribe el método `saludar`, sustituyendo por completo su comportamiento. Ilustra el funcionamiento del polimorfismo creando un array de `Soldados` de dos

tipos y luego recorriéndolo empleando referencias de tipo `Soldado` y llamando a `saludar`.

Respuesta

```
class Soldado {
    public void saluda() {
        System.out.println("¡Soldado saludando!");
    }
}

class Zapador extends Soldado {
    @Override
    public void saluda() {
        System.out.println("¡Zapador listo para la acción!");
    }
}

class Artillero extends Soldado {
    @Override
    public void saluda() {
        System.out.println("¡Artillero preparado para disparar!");
    }
}

public class Main {
    public static void main(String[] args) {
        Soldado[] soldados = {new Zapador(), new Artillero()};

        for (Soldado s : soldados) {
            s.saluda();
        }
    }
}
```

En este ejemplo, se crea un arreglo de `Soldado` que contiene instancias de `Zapador` y `Artillero`. Al recorrer el arreglo y llamar al método `saluda`, se ejecuta la implementación específica de cada subclase gracias al polimorfismo.

4. Si sobreescribo un método, ¿puedo invocar el método base para trabajar a partir de su resultado? Haz que zapador cambie ligeramente la forma de saludar, que salude de forma normal, tal cual hace el soldado base, pero que además añada un "ZAPADOR A SUS ORDENES" ¿qué palabra clave del lenguaje has usado para invocar al método de la clase base?

Respuesta

Sí, al sobreescribir un método en Java, es posible invocar el método de la clase base utilizando la palabra clave `super`. Esto permite reutilizar el comportamiento definido en la clase base y extenderlo con lógica adicional en la subclase.

```
class Zapador extends Soldado {
    @Override
    public void saluda() {
        super.saluda(); // Invoca el método de la clase base
        System.out.println("ZAPADOR A SUS ORDENES");
    }
}
```

En este caso, el método `saluda` de `Zapador` primero llama al método `saluda` de la clase base `Soldado` utilizando `super.saluda()`. Luego, añade su propio mensaje personalizado. Esto demuestra cómo se puede extender el comportamiento de un método base al sobreescribirlo.

5. Al sobreescribir un método en Java, ¿qué restricciones existen sobre los tipos de los parámetros y el tipo de retorno? ¿Qué diferencia hay entre sobreescritura (*overriding*) y sobrecarga

(*overloading*)? ¿Para qué sirve la anotación `@Override` y por qué es recomendable usarla siempre?

Respuesta

Al sobreescribir un método en Java, los parámetros deben coincidir exactamente en número, tipo y orden con los del método en la clase base. El tipo de retorno puede ser el mismo o un subtipo del tipo de retorno del método base (esto se conoce como covarianza en el tipo de retorno). Además, el nivel de acceso del método sobreescrito no puede ser más restrictivo que el del método original. Por ejemplo, si el método base es `protected`, el método sobreescrito no puede ser `private`.

La sobreescritura (*overriding*) ocurre cuando una subclase redefine un método de su clase base con la misma firma. Esto permite personalizar el comportamiento heredado. Por otro lado, la sobrecarga (*overloading*) consiste en definir múltiples métodos con el mismo nombre pero diferentes firmas (distinto número o tipo de parámetros) dentro de la misma clase. La sobrecarga no está relacionada con el polimorfismo, ya que se resuelve en tiempo de compilación.

La anotación `@Override` se utiliza para indicar explícitamente que un método está siendo sobreescrito. Es recomendable usarla siempre porque ayuda al compilador a detectar errores, como cuando el método no coincide exactamente con el de la clase base debido a un error tipográfico. Además, mejora la legibilidad del código al dejar claro que el método redefine uno existente en la superclase.

6. Entonces, cuando se estudia Java, ¿se emplea el polimorfismo desde el principio? Por ejemplo, sobreescribiendo `toString` o sobreescribiendo `equals`, ¿ya estoy usando polimorfismo?

Respuesta

Sí, el polimorfismo se emplea desde el principio al estudiar Java, incluso en casos básicos como la sobreescritura de métodos como `toString` o `equals`. Estos métodos están definidos en la clase base `Object`, que es la superclase de todas las clases en Java. Al sobreescribirlos, se está utilizando polimorfismo porque las llamadas a estos métodos se resuelven dinámicamente en tiempo de ejecución según el tipo real del objeto.

Por ejemplo, al sobreescribir `toString`, se puede personalizar la representación en forma de cadena de un objeto. Esto permite que, al imprimir un objeto, se invoque automáticamente la implementación específica de `toString` de su clase. De manera similar, al sobreescribir `equals`, se puede definir un criterio personalizado para comparar objetos, lo que también aprovecha el polimorfismo.

7. ¿Qué es una "clase abstracta"? ¿Qué es un "método abstracto"? ¿Puedo crear instancias de una clase abstracta? Pongamos un ejemplo en Java: Redefinamos `Soldado`, hagamos que, además del método `saluda` que ya tenía, tenga un método `atacar`, que sea abstracto y que cada tipo de soldado haga su acción cuando se le pida atacar. ¿Donde debemos poner `abstract`?

Respuesta

Una clase abstracta es una clase que no puede ser instanciada directamente y que está diseñada para ser extendida por otras clases. Puede contener métodos concretos (con implementación) y métodos abstractos (sin implementación). Un método abstracto es aquel que se declara sin cuerpo, es decir, sin una implementación, y obliga a las subclases a proporcionar su propia versión del método.

El modificador `abstract` se utiliza para declarar tanto clases como métodos abstractos. En el caso de `Soldado`, el método `atacar` puede ser abstracto, lo que significa que cada subclase debe implementar su propia lógica para este método.

```java
abstract class Soldado {
    public void saluda() {
        System.out.println("¡Soldado saludando!");
    }

    public abstract void atacar(); // Método abstracto
}

class Zapador extends Soldado {
    @Override
    public void atacar() {
        System.out.println("Zapador desactivando minas.");
    }
}

class Artillero extends Soldado {
    @Override
    public void atacar() {
        System.out.println("Artillero disparando cañones.");
    }
}

public class Main {
    public static void main(String[] args) {
        Soldado zapador = new Zapador();
        Soldado artillero = new Artillero();

        zapador.atacar();
        artillero.atacar();
    }
}
```

En este ejemplo, `Soldado` es una clase abstracta con un método abstracto `atacar`. Las subclases `Zapador` y `Artillero` implementan este método con su propio comportamiento.

8. ¿Qué efecto tiene la palabra clave `final` sobre métodos y clases en Java? ¿Cómo se relaciona con el

polimorfismo? ¿Conoces algún ejemplo de clase `final` en la propia API estándar de Java?

Respuesta

La palabra clave `final` en Java tiene diferentes efectos dependiendo de dónde se utilice. Cuando se aplica a un método, indica que el método no puede ser sobreescrito por las subclases. Esto significa que el comportamiento del método queda fijo y no puede ser modificado. Cuando se aplica a una clase, indica que la clase no puede ser heredada, es decir, no se pueden crear subclases de ella.

En relación con el polimorfismo, el uso de `final` en métodos o clases limita su flexibilidad. Por ejemplo, si un método es `final`, no se puede personalizar su comportamiento en las subclases, lo que restringe el uso del polimorfismo. Sin embargo, esto puede ser útil para garantizar la seguridad o la consistencia del comportamiento en ciertas partes del código.

Un ejemplo de una clase `final` en la API estándar de Java es `String`. La clase `String` es `final` para garantizar su inmutabilidad y evitar que su comportamiento sea alterado mediante la herencia.

9. En Java, qué son las "interfaces"? ¿Son como clases abstractas? ¿Una clase puede implementar más de una interfaz?

Respuesta

En Java, una interfaz es un tipo de contrato que define un conjunto de métodos que una clase debe implementar. A diferencia de las clases abstractas, las interfaces no pueden contener implementaciones de métodos (hasta Java 7). A partir de Java 8, las interfaces pueden incluir métodos con implementación predeterminada (`default`) y métodos estáticos.

Aunque las interfaces y las clases abstractas comparten similitudes, como el hecho de que no se pueden instanciar directamente, tienen diferencias clave. Por ejemplo, una clase puede implementar múltiples interfaces, pero solo puede heredar de una única clase (abstracta o no). Esto permite a las interfaces proporcionar una forma de herencia múltiple en Java.

```java
interface Volador {
    void volar();
}

interface Nadador {
    void nadar();
}

class SuperHeroe implements Volador, Nadador {
    @Override
    public void volar() {
        System.out.println("Volando por los cielos.");
    }

    @Override
    public void nadar() {
        System.out.println("Nadando en el océano.");
    }
}
```

En este ejemplo, la clase `SuperHeroe` implementa dos interfaces, `Volador` y `Nadador`, demostrando que una clase puede implementar múltiples interfaces.

10. Vamos a poner un ejemplo nuevo con polimorfismo. Queremos implementar una clase `Punto`, con un método `calcularDistanciaA`, que permite calcular la distancia a otro `Punto`. Sin embargo, como queremos trabajar con puntos 2D y 3D, haz que ese método sea abstracto y haya dos implementaciones de ese cálculo de distancia. Emplea `instanceof` y *downcasting* para verificar que se recibe un punto compatible y poder calcular correctamente la distancia siempre entre puntos del mismo subtipo. Aprovecha este diseño para crear

ahora una clase `Linea`, que acepta `Punto`, sin saber de qué tipo es, y es capaz de dar su longitud independientemente de las dimensiones de sus puntos (las cuales desconoce).

Respuesta

```java
abstract class Punto {
    public abstract double calcularDistanciaA(Punto otro);
}

class Punto2D extends Punto {
    private double x, y;

    public Punto2D(double x, double y) {
        this.x = x;
        this.y = y;
    }

    @Override
    public double calcularDistanciaA(Punto otro) {
        if (otro instanceof Punto2D) {
            Punto2D p = (Punto2D) otro;
            return Math.sqrt(Math.pow(this.x - p.x, 2) + Math.pow(this.y - p.y, 2));
        }
        throw new IllegalArgumentException("Tipo incompatible de punto.");
    }
}

class Punto3D extends Punto {
    private double x, y, z;

    public Punto3D(double x, double y, double z) {
        this.x = x;
        this.y = y;
        this.z = z;
    }

    @Override
    public double calcularDistanciaA(Punto otro) {
```

```
        if (otro instanceof Punto3D) {
            Punto3D p = (Punto3D) otro;
            return Math.sqrt(Math.pow(this.x - p.x, 2) + Math.pow(this.y - p.y, 2) + Math.pow(tl
        }
        throw new IllegalArgumentException("Tipo incompatible de punto.");
    }
}

class Linea {
    private Punto inicio, fin;

    public Linea(Punto inicio, Punto fin) {
        this.inicio = inicio;
        this.fin = fin;
    }

    public double calcularLongitud() {
        return inicio.calcularDistanciaA(fin);
    }
}
```

En este ejemplo, `Punto` es una clase abstracta con un método abstracto `calcularDistanciaA`. Las clases `Punto2D` y `Punto3D` implementan este método para calcular la distancia entre puntos del mismo tipo. La clase `Linea` utiliza polimorfismo para calcular su longitud sin importar si los puntos son 2D o 3D.

11. ¿Qué es la "herencia de interfaces" en Java? ¿Existe "herencia múltiple de interfaces"? Pon un ejemplo de una interfaz `Fichero` que tenga un método para leer su contenido en forma de `String` y luego dicha interfaz sea extendida por otra que sea

`FicheroEscribible` que permita enviar contenido e incluso eliminar el fichero.

Respuesta

La herencia de interfaces en Java ocurre cuando una interfaz extiende otra interfaz. Esto permite que una interfaz herede los métodos de otra y añada nuevos métodos. A diferencia de las clases, las interfaces pueden extender múltiples interfaces, lo que se conoce como herencia múltiple de interfaces.

```java
interface Fichero {
    String leerContenido();
}

interface FicheroEscribible extends Fichero {
    void escribirContenido(String contenido);
    void eliminar();
}

class ArchivoTexto implements FicheroEscribible {
    private String contenido = "";

    @Override
    public String leerContenido() {
        return contenido;
    }

    @Override
    public void escribirContenido(String contenido) {
        this.contenido = contenido;
    }

    @Override
    public void eliminar() {
        contenido = "";
    }
}
```

En este ejemplo, `FicheroEscribible` extiende `Fichero`, añadiendo métodos para escribir y eliminar contenido. La clase `ArchivoTexto` implementa esta interfaz, proporcionando una implementación

concreta de todos los métodos.